

Flujo CI/CD para Django

Integración y Despliegue Continuo con GitHub Actions y Docker

Pipeline CI / Build / Deploy

Proyecto: Sistema de Gestión de Tareas (Django + PostgreSQL)

Repositorio: github.com/Eneritzch/P3_GCS

Fecha: 18 de junio de 2026

1. Introducción y objetivo

Este documento describe el **pipeline de CI/CD** construido para una aplicación web desarrollada en **Django** con base de datos **PostgreSQL**. El objetivo es automatizar tres tareas que tradicionalmente se hacían a mano:

- **CI (Integración Continua):** ejecutar las pruebas automáticas cada vez que se sube código, para detectar errores temprano.
- **Build:** empaquetar la aplicación en una imagen Docker reproducible.
- **CD (Despliegue Continuo, simulado):** publicar/entregar la imagen y levantarla con Docker Compose para validarla.

2. Descripción del proyecto

La aplicación es un gestor de tareas (CRUD) con una app funcional. Estructura:

```
P3_GCS/
├── manage.py           # utilidad de administración de Django
├── requirements.txt    # dependencias (Django, psycopg2, gunicorn)
├── project/           # configuración del proyecto
│   ├── settings.py    # BD vía variables de entorno (Postgres/SQLite)
│   ├── urls.py · wsgi.py · asgi.py
│   └── app/           # aplicación funcional
│       ├── models.py (Tarea) · views.py · forms.py · urls.py
│       ├── admin.py · tests.py (11 pruebas)
│       └── templates/app/lista_tareas.html
├── Dockerfile         # imagen del servicio web (Python 3.10)
├── docker-compose.yml  # orquesta web (Django) + db (PostgreSQL)
├── entrypoint.sh      # espera BD → migra → arranca Gunicorn
└── .github/workflows/ci.yml # pipeline CI/CD
```

3. Diagrama del flujo CI/CD



Desarrollador

Escribe código y ejecuta `git push` a la rama `main` / `ci-setup`



GitHub — disparador del workflow

El evento `push` / `pull_request` activa GitHub Actions (`ci.yml`)



✓ JOB 1 · test (Integración Continua)

Levanta servicio PostgreSQL 15 → instala Python 3.10 + dependencias → `migrate` → `python manage.py test`

▼ solo si los tests pasan (needs: test)

🚀 JOB 2 · build (Construcción)

`docker build` → `docker save` → sube artefacto `imagen-docker-django` → (simulado) push a DockerHub



🚀 Despliegue simulado

Carpeta externa → `git clone` / `docker load` → `docker compose up` → validación en navegador (HTTP 200)

4. Explicación del archivo de GitHub Actions (`ci.yml`)

4.1. Disparadores (`on`)

```
on:
  push:
    branches: ["main", "ci-setup"]
  pull_request:
    branches: ["main"]
  workflow_dispatch: # ejecución manual desde la pestaña Actions
```

El pipeline se ejecuta automáticamente al hacer `push` a `main` o `ci-setup`, al abrir un *pull request* hacia `main`, o manualmente.

4.2. Job 1 — `test` (CI)

Elemento	Función
<code>services: postgres</code>	Levanta un contenedor PostgreSQL 15 con un <i>healthcheck</i> para que los tests usen una BD real.
<code>env</code>	Define las variables que lee <code>settings.py</code> (credenciales y host de la BD).
<code>actions/checkout</code>	Clona el repositorio en el runner.
<code>setup-python 3.10</code>	Instala Python 3.10 con caché de pip.
<code>pip install -r requirements.txt</code>	Instala dependencias.
<code>manage.py check / migrate</code>	Valida el proyecto y aplica migraciones.
<code>manage.py test</code>	Ejecuta las 11 pruebas. Si una falla, el pipeline se detiene.

4.3. Job 2 — `build` (Construcción + entrega)

Elemento	Función
<code>needs: test</code>	El build solo corre si los tests pasaron . Garantiza que nunca se empaqueta código roto.
<code>setup-buildx</code>	Prepara el motor de construcción de Docker.
<code>docker build</code>	Construye la imagen con dos etiquetas: <code>latest</code> y el <code>SHA</code> del commit (trazabilidad).
<code>docker save</code>	Empaqueta la imagen en <code>gcs-web-image.tar</code> .
<code>upload-artifact</code>	Sube el <code>.tar</code> como artefacto descargable desde GitHub.

Push a DockerHub (simulado)	Imprime los comandos que se ejecutarían en un despliegue real. El bloque real queda comentado y se activa creando los <i>secrets</i> <code>DOCKERHUB_USER</code> y <code>DOCKERHUB_TOKEN</code> .
--------------------------------	---

5. Contenedorización con Docker

5.1. Dockerfile

Imagen base `python:3.10-slim`; instala dependencias en una capa separada (mejor caché), copia el proyecto y define un `ENTRYPOINT` que prepara el entorno antes de arrancar el servidor de aplicación **Gunicorn**.

5.2. entrypoint.sh

Espera a que PostgreSQL responda (`pg_isready`), aplica migraciones, recolecta archivos estáticos y finalmente ejecuta el comando del contenedor.

5.3. docker-compose.yml

Servicio	Descripción
<code>db</code>	PostgreSQL 15 con volumen persistente (<code>postgres_data</code>) y <i>healthcheck</i> .
<code>web</code>	Aplicación Django (Gunicorn). Depende de que <code>db</code> esté <i>healthy</i> . Mapea <code>8000:8000</code> .

Mapeo de puertos: `"8000:8000"` conecta el puerto 8000 del equipo con el 8000 del contenedor; por eso `http://localhost:8000` muestra la app que corre dentro de Docker.

6. Despliegue simulado

Se simuló un servidor de producción en la carpeta externa `C:\despliegue_gcs`:

```
# 1) Carpeta externa que simula el servidor
mkdir C:\despliegue_gcs && cd C:\despliegue_gcs

# 2) Descargar el proyecto
git clone https://github.com/Eneritzch/P3_GCS.git && cd P3_GCS

# 3) Levantar con Docker Compose
docker compose up -d --build

# 4) Validar
docker compose ps          # contenedores Up
# Navegador → http://localhost:8000 (HTTP 200, app funcional)
```

Resultado: contenedores `gcs_web` y `gcs_db` en estado **Up**, respuesta **HTTP 200** y CRUD operativo. Los pasos completos están en `deploy-notes.md`.

7. Recomendaciones finales

- **Gestión de secretos:** no incluir `SECRET_KEY` ni contraseñas en el código ni en `compose` ; usar *GitHub Secrets* y variables de entorno del servidor.
- **DEBUG en producción:** mantener `DJANGO_DEBUG=False` y configurar `ALLOWED_HOSTS` con el dominio real.
- **DockerHub real:** activar el bloque comentado del job `build` con `docker/login-action` y `build-push-action` para publicar la imagen automáticamente.
- **Calidad de código:** añadir *linters* (flake8/black) y cobertura de pruebas (`coverage`) como pasos del job `test` .
- **Entornos:** separar configuraciones de desarrollo y producción y usar un servidor de estáticos (WhiteNoise o Nginx).
- **Despliegue real:** sustituir la simulación por un destino real (servidor, VPS o servicio cloud) usando SSH o un runner autoalojado.

8. Conclusiones

El pipeline implementado cubre el ciclo completo de **Integración y Entrega Continua**: cada cambio subido a GitHub se prueba automáticamente, y solo si las pruebas pasan se construye y empaqueta la imagen Docker, lista para desplegarse. La contenedorización garantiza que la aplicación corra igual en cualquier entorno, eliminando el clásico problema de "en mi máquina sí funciona".

Anexo · Checklist de entregables

Sesión	Entregable	Estado
1	Proyecto Django (manage.py, project/, app/, requirements.txt) + tests	✓
1	Rama <code>ci-setup</code> + <code>ci.yml</code> ejecutando tests	✓
2	<code>Dockerfile</code> y <code>docker-compose.yml</code> funcionales	✓
3	Build de imagen + artefacto en Actions	✓
3	Despliegue simulado + <code>deploy-notes.md</code>	✓
4	Documento técnico (este PDF) + repositorio actualizado	✓